



Architecture Decision Record

CoExist Alert

Code with Cisco | Silver Flag CSR Challenge | Mission 3: Human-Animal Coexistence

Decision Summary

CoExist Alert is a proof of concept for human-wildlife conflict response at forest-village boundaries and rail crossings. The central architecture decision is to make the warning network trustworthy: detect at the edge, confirm before alerting, route warnings to the right people, prove delivery, and escalate if nobody responds. The proof of concept is honest about simulation while keeping the ingest, alerting, Webex, auth, data, export, and response loops real.

Context

The product must be judge-clonable, demoable in minutes, and clear about which Cisco layers are production architecture versus proof-of-concept simulation. Field hardware, sirens, villager phones, and rail/control-room endpoints are simulated. The backend, data model, health monitoring, event confirmation, responder workflow, Webex delivery, auth/RBAC, Duo integration path, and Splunk-style export are implemented.

Accepted Decisions

1. One persistent Next.js application

The system uses one Next.js 16 App Router application for pages, APIs, simulator, persistence access, server-sent events, and production runtime. Escalation timers and heartbeat-timeout checks need a long-lived process, so the deployment runs as a persistent Node service instead of a serverless function set. A reviewer can run everything with `docker compose up` (nothing but Docker required), or on Node 20+ with `npm install`, `npm run setup`, and `npm run dev`.

2. Edge classification stays outside this platform

In production, Meraki MV cameras and field sensors perform classification and publish object-detection style payloads. The platform consumes `classification`, `confidence`, and `snapshot` inputs, validates them, suppresses false positives, and coordinates warnings. This keeps the cloud app focused on reliability and response rather than adding an extra model runtime.

3. Deterministic domain core

Health transitions, event confirmation, cascade planning, targeting, escalation, ownership, and metrics are implemented as deterministic TypeScript logic with tests. A weak single signal never alerts; corroboration must be cross-source and inside the window; rail-crossing events include control-room targeting; missed heartbeats create blind-spot alerts.

4. SQLite and Drizzle for zero-setup persistence

SQLite keeps the clone path lightweight while still giving the demo durable, auditable state. Drizzle provides typed schema and migrations for nodes, signals, events, alerts, responses, outages, responders, and users, with a clear route to Postgres if the project grows beyond the proof of concept. The seeded month of demo history is anchored to the install date, so analytics are fully populated whenever the repository is cloned.

5. REST actions and SSE dashboards

Mutations use HTTP endpoints: ingest detection, ingest heartbeat, demo controls, responder actions, Webex callbacks, auth, and export. Dashboards use server-sent events for live updates and refetch snapshots after reconnect. This gives an operations-console feel without unnecessary socket infrastructure.

6. Cisco products carry real architectural roles

Meraki MV/MT/Cisco Spaces define the sensing layer; ThousandEyes and Splunk shape health semantics, analytics, and export; Webex is a real guard-alert channel; Meraki MG/Catalyst explain the backhaul; Duo/Secure Access/Umbrella define the secure production path. Every simulated surface is labelled **SIMULATED**; real Webex sends show **LIVE**.

7. The simulator uses the real ingest boundary

Demo controls drive the same ingest APIs that real field integrations would call. They do not directly mutate UI state. This keeps the visible demo backed by the same records, delivery states, timelines, and analytics as the production path.

8. Real authentication with deliberately open demo accounts

Every console sits behind real sign-in: script-hashed passwords, signed session cookies, and role-gating middleware on every page and mutating API. For review, all five roles — including the operations admin that drives the demo panel — are one-click demo accounts with a shared, displayed password. That is deliberate: reviewers explore the whole proof of concept without gatekeeping, and switching roles demonstrates the RBAC boundaries live. Production swaps these for directory-backed accounts with Cisco Duo MFA step-up, already implemented and active when configured.

Consequences

The architecture optimizes for clarity, trust, and reviewability. It avoids live hardware dependency, a separate model service, and paid cloud setup, but still demonstrates a real backend, durable state, auth/RBAC, Webex integration, health monitoring, alert escalation, export, and a repeatable seed. The main tradeoff is that sensing hardware and several channel endpoints are simulated; the product handles that visibly and ties each simulated layer to its Cisco production counterpart. Scaling beyond the proof of concept is a swap of endpoints, not a redesign: Postgres behind the same repositories, real sensor fleets behind the same ingest contract, and directory-backed identity behind the same session middleware.